

Module 19 Week 21 Required Assignment

1. Generative AI for code generation in a full stack project

Implementation steps:

1. Pick an integration point - an IDE assistant (e.g. an LLM-powered autocomplete/chat plugin) for day-to-day coding, and/or an in-app "generate code" feature (e.g. a low-code admin panel that asks an LLM to draft a CRUD endpoint from a natural-language description).
2. Give the model context - feed it the relevant existing code (schema, existing route conventions, style guide) as part of the prompt so the generated code matches the project's actual patterns instead of generic boilerplate.
3. Constrain the output format - ask for a specific file/function shape (e.g. "return only the Express route handler, no explanation") so the output can be inserted directly or reviewed as a diff.
4. Validate before trusting - run the project's linter, type checker and test suite against the generated code, and require a human review/approval step before it's merged, the same as any other change.
5. Iterate - feed test/lint failures back to the model as follow-up context so it can correct its own output.

Benefits: much faster scaffolding of repetitive code (CRUD routes, boilerplate components, test stubs), lower ramp-up time for less familiar parts of a stack, and a natural-language interface for less technical teammates to prototype ideas.

Challenges: generated code can look plausible but be subtly wrong (wrong edge cases, security issues like missing input validation, or outdated library APIs), it can drift from the project's actual conventions if not given enough context, and over-reliance on it can erode a team's understanding of code they didn't actually write. All of this is why validation and review (step 4 above) has to stay in the loop rather than being treated as optional.

2. Converting natural language requests into SQL queries or API calls

Generative AI (an LLM, typically fine-tuned or prompted with the target schema/API spec) can translate a plain-English request like `"show me all students who enrolled in the AI programme after January"` directly into a query such as:

```
SELECT * FROM students
WHERE program = 'AI' AND enrolled_date > '2026-01-01';
```

The same idea applies to API calls - `"enroll user 42 in course 7"` could be translated into a POST /enroll call with `{"userId": 42, "courseId": 7}`, by giving the model the API's OpenAPI spec as context so it knows the exact endpoint shape and required fields.

Implications:

- Database management - the generated query must be validated (e.g. run through a query parser, restricted to read-only/parameterised statements) before execution, since a malformed or maliciously-crafted natural-language input could otherwise produce a destructive query (a text-to-SQL analogue of SQL injection).
- API design - APIs intended to be driven this way benefit from being well-documented (OpenAPI specs), having consistent, predictable naming, and returning clear validation errors, since the LLM's translation

quality depends entirely on how unambiguous the underlying schema/API is.

3. Fine-tuning a pre-trained language model

Fine-tuning starts from a large model that has already learned general language patterns from a broad, generic corpus, then continues training it on a smaller, task- or domain-specific dataset (e.g. a company's own support tickets, or a specific coding style), adjusting the model's weights so its outputs are better suited to that narrower task.

Advantages over training from scratch:

- Far less data and compute needed - the model already "knows" grammar, general world knowledge and reasoning patterns; fine-tuning only needs to adapt behaviour, not build language understanding from zero.
- Much faster and cheaper - hours/days on a modest dataset instead of the massive compute budgets (and datasets) required to pre-train a foundation model.
- Better performance on the specific task, since the model's weights are specialised toward the target domain's vocabulary, tone and typical requests, while still retaining the general capabilities learned during pre-training.

4. Prompt engineering

Prompt engineering is the practice of carefully designing the input (instructions, context, examples, formatting) given to a large language model in order to reliably get the output you actually want, without changing the model's underlying weights. It matters because LLMs are highly sensitive to how a request is phrased - a vague prompt tends to produce a vague or inconsistent answer, while a well-structured one that specifies the role, format, constraints and examples produces far more reliable, on-target output.

Example - an effective prompt for a text generation task (writing a product description):

> "You are an e-commerce copywriter. Write a 2-sentence product description for a stainless steel water bottle, 1L capacity, insulated for 12 hours. Tone: friendly and concise. Do not mention price. Output only the description, no heading."

This works because it specifies the role (copywriter), the input facts (capacity, insulation), the constraints (2 sentences, no price), the tone, and the exact output format (no heading, description only) - leaving very little room for the model to guess wrong about what's expected.